

```
clc; clear; close all;
```

```
%% ===== 1. READ IMAGE =====
```

```
original_img = imread('C:\Users\ASUS\Documents\DIP PROJECT\Assignment.png');
```

```
if size(original_img,3) == 3
```

```
    original_img = rgb2gray(original_img);
```

```
end
```

```
original_img = double(original_img);
```

```
num_pixels = numel(original_img);
```

```
figure; imshow(uint8(original_img)); title('Original Image');
```

```
%% ===== 2. WATERMARK =====
```

```
wm_text = '23FE10ECE00100';
```

```
wm_bin = reshape(dec2bin(wm_text,8).-'0',[],1);
```

```
wm_length = length(wm_bin);
```

```
%% ===== 3. KEY =====
```

```
key = 100;
```

```
key_bit = mod(key,2);
```

```
%% ===== 4. POSITIONS =====
```

```
rng(key);
```

```
positions = randperm(num_pixels, wm_length);
```

```
%% ===== 5. ENCRYPT =====
```

```
wm_enc = xor(wm_bin, key_bit);
```

```
%% ===== 6. EMBEDDING =====
```

```
img_flat = original_img(:);
```

```
for i = 1:wm_length
```

```
    pos = positions(i);
```

```
    if wm_enc(i) == 1
```

```
        if mod(img_flat(pos),2) == 0
```

```
            img_flat(pos) = img_flat(pos) + 1;
```

```
        end
```

```
    else
```

```
        if mod(img_flat(pos),2) == 1
```

```
            img_flat(pos) = img_flat(pos) - 1;
```

```
        end
```

```
    end
```

```
end
```

```
watermarked_img = uint8(reshape(img_flat, size(original_img)));
```

```
figure;
```

```
imshow(watermarked_img);
```

```
title('Watermarked Image');
```

```
imwrite(watermarked_img,'watermarked.png');
```

```
%% ===== 7. PSNR (ORIGINAL vs WM) =====
```

```
psnr_value = psnr(watermarked_img, uint8(original_img));
```

```
figure;
```

```
bar(psnr_value);
```

```
set(gca,'XTickLabel',{'Original vs Watermarked'});
```

```
ylabel('PSNR (dB)');
```

```
title('PSNR Comparison');
```

```
text(1,psnr_value,num2str(psnr_value,'% .2f'),'HorizontalAlignment','center');
```

```
%% ===== SSIM (ORIGINAL vs WATERMARKED) =====
```

```
ssim_value = ssim(watermarked_img, uint8(original_img));
```

```
disp(['SSIM (Original vs Watermarked): ', num2str(ssim_value)]);
```

```
figure;
```

```
bar(ssim_value);
```

```
set(gca,'XTickLabel',{'Original vs Watermarked'});
```

```
ylabel('SSIM Index');
```

```
title('SSIM Comparison');
```

```
text(1, ssim_value, num2str(ssim_value,'% .4f'), ...
```

```
    'HorizontalAlignment','center','VerticalAlignment','bottom');
```

```
%% ===== 8. BER + PSNR UNDER ATTACK =====
```

```
noise_levels = 0:0.01:0.05;
```

```
ber_gaussian = zeros(size(noise_levels));
```

```

ber_sp = zeros(size(noise_levels));
ber_blur = zeros(size(noise_levels));

psnr_g = zeros(size(noise_levels));
psnr_sp = zeros(size(noise_levels));
psnr_blur = zeros(size(noise_levels));

for i = 1:length(noise_levels)

    %% Gaussian
    g_img = imnoise(watermarked_img,'gaussian',0,noise_levels(i));
    ber_gaussian(i) = computeBER(g_img, wm_bin, positions, key_bit);
    psnr_g(i) = psnr(g_img, watermarked_img);

    %% Salt & Pepper
    sp_img = imnoise(watermarked_img,'salt & pepper',noise_levels(i));
    ber_sp(i) = computeBER(sp_img, wm_bin, positions, key_bit);
    psnr_sp(i) = psnr(sp_img, watermarked_img);

    %% Blur (increasing)
    h = fspecial('average',[3+2*i 3+2*i]);
    blur_img = imfilter(watermarked_img,h,'replicate');
    ber_blur(i) = computeBER(blur_img, wm_bin, positions, key_bit);
    psnr_blur(i) = psnr(blur_img, watermarked_img);

end

%% ===== BER GRAPH =====

```

```

figure;
plot(noise_levels, ber_gaussian,'-o','LineWidth',2); hold on;
plot(noise_levels, ber_sp,'-s','LineWidth',2);
plot(noise_levels, ber_blur,'-^','LineWidth',2);
xlabel('Noise Level');
ylabel('BER');
title('BER Comparison');
legend('Gaussian','Salt & Pepper','Blur');
grid on;

```

```

%% ===== PSNR GRAPH =====

```

```

figure;
plot(noise_levels, psnr_g,'-o','LineWidth',2); hold on;
plot(noise_levels, psnr_sp,'-s','LineWidth',2);
plot(noise_levels, psnr_blur,'-^','LineWidth',2);
xlabel('Noise Level');
ylabel('PSNR (dB)');
title('PSNR under Attacks');
legend('Gaussian','Salt & Pepper','Blur');
grid on

```

```

%% ===== SSIM UNDER ATTACKS =====

```

```

ssim_g = zeros(size(noise_levels));
ssim_sp = zeros(size(noise_levels));
ssim_blur = zeros(size(noise_levels));

for i = 1:length(noise_levels)

```

```

% Gaussian

g_img = imnoise(watermarked_img,'gaussian',0,noise_levels(i));
ssim_g(i) = ssim(g_img, watermarked_img);


% Salt & Pepper

sp_img = imnoise(watermarked_img,'salt & pepper',noise_levels(i));
ssim_sp(i) = ssim(sp_img, watermarked_img);


% Blur

h = fspecial('average',[3+2*i 3+2*i]);
blur_img = imfilter(watermarked_img,h,'replicate');
ssim_blur(i) = ssim(blur_img, watermarked_img);


end


%% ===== SSIM GRAPH =====

figure;

plot(noise_levels, ssim_g,'-o','LineWidth',2); hold on;
plot(noise_levels, ssim_sp,'-s','LineWidth',2);
plot(noise_levels, ssim_blur,'-^','LineWidth',2);


xlabel('Noise Level');
ylabel('SSIM Index');
title('SSIM under Attacks');


legend('Gaussian','Salt & Pepper','Blur');
grid on;

```

```
disp('✅ SSIM Analysis Done');
```

```
%% ===== 9. EXTRACTION (NO ATTACK) =====
```

```
[txt_clean, img_clean] = extractWM(watermarked_img, positions, wm_length, key_bit);
```

```
disp('Extracted Text (No Attack):');
```

```
disp(txt_clean);
```

```
figure;
```

```
imshow(img_clean);
```

```
title('Extracted Watermark (No Attack)');
```

```
%% ===== 10. ATTACK VISUAL + EXTRACTION =====
```

```
% Gaussian
```

```
g_img = imnoise(watermarked_img,'gaussian',0,0.01);
```

```
figure; imshow(g_img); title('Gaussian Noise');
```

```
[txt_g, img_g] = extractWM(g_img, positions, wm_length, key_bit);
```

```
disp('Gaussian Text:'); disp(txt_g);
```

```
figure; imshow(img_g); title('WM Gaussian');
```

```
% Salt & Pepper
```

```
sp_img = imnoise(watermarked_img,'salt & pepper',0.02);
```

```
figure; imshow(sp_img); title('Salt & Pepper');
```

```
[txt_sp, img_sp] = extractWM(sp_img, positions, wm_length, key_bit);
```

```
disp('SP Text:'); disp(txt_sp);
```

```
figure; imshow(img_sp); title('WM SP');
```

```

% Blur

h = fspecial('gaussian',[5 5],1);
blur_img = imfilter(watermarked_img,h);
figure; imshow(blur_img); title('Blur');

[txt_b, img_b] = extractWM(blur_img, positions, wm_length, key_bit);
disp('Blur Text:'); disp(txt_b);
figure; imshow(img_b); title('WM Blur');

%% ===== 11. CORRELATION =====

disp('Running Correlation Analysis...');

% ----- Correct Key -----

[~,~, wm_correct_bits] = extractWM_bits(watermarked_img, positions, wm_length,
key_bit);

corr_correct = corr2(double(wm_bin), double(wm_correct_bits));

disp(['Correlation (Correct Key): ', num2str(corr_correct)]);

% ----- Wrong Key -----

wrong_key = key + 1;

rng(wrong_key);
positions_wrong = randperm(num_pixels, wm_length);

wrong_key_bit = mod(wrong_key,2);

```

```
[~, ~, wm_wrong_bits] = extractWM_bits(watermarked_img, positions_wrong,
wm_length, wrong_key_bit);
```

```
corr_wrong = corr2(double(wm_bin), double(wm_wrong_bits));
```

```
disp(['Correlation (Wrong Key): ', num2str(corr_wrong)]);
```

```
%% ----- GRAPH -----
```

```
figure;
```

```
bar([corr_correct corr_wrong]);
```

```
set(gca,'XTickLabel',{'Correct Key','Wrong Key'});
```

```
ylabel('Correlation Coefficient');
```

```
title('Correlation Analysis');
```

```
grid on;
```

```
text(1, corr_correct, num2str(corr_correct,'%0.2f'), ...
```

```
    'HorizontalAlignment','center','VerticalAlignment','bottom');
```

```
text(2, corr_wrong, num2str(corr_wrong,'%0.2f'), ...
```

```
    'HorizontalAlignment','center','VerticalAlignment','bottom');
```

```
%% ===== FUNCTIONS =====
```

```
function ber = computeBER(attacked_img, wm_original, positions, key_bit)
```

```

% Flatten image

img_flat = double(attacked_img(:));

wm_length = length(wm_original);

% Initialize extracted bits

wm_ext = zeros(wm_length,1);

% Extract bits using EVEN-ODD rule
for j = 1:wm_length
    if mod(round(img_flat(positions(j))),2) == 1
        wm_ext(j) = 1;
    else
        wm_ext(j) = 0;
    end
end

% Decrypt watermark

wm_dec = xor(wm_ext, key_bit);

% Compute BER

ber = sum(wm_original ~= wm_dec) / wm_length;

end

function [text_out, wm_img, wm_bits] = extractWM_bits(input_img, positions,
wm_length, key_bit)

% Flatten image

```

```

img_flat = double(input_img(:));

% Initialize extracted bits
wm_ext = zeros(wm_length,1);

% Extract watermark bits
for i = 1:wm_length
    if mod(round(img_flat(positions(i))),2) == 1
        wm_ext(i) = 1;
    else
        wm_ext(i) = 0;
    end
end

% Decrypt bits
wm_bits = xor(wm_ext, key_bit);

% Convert to TEXT
wm_matrix = reshape(wm_bits,8,[]);
ascii_vals = bin2dec(char(wm_matrix + '0'));
text_out = char(ascii_vals);

% Convert to IMAGE
wm_img = reshape(wm_bits, 8, []);
wm_img = uint8(wm_img * 255);

end

function [text_out, wm_img] = extractWM(input_img, positions, wm_length, key_bit)

```

```
[text_out, wm_img, ~] = extractWM_bits(input_img, positions, wm_length, key_bit);
```

```
end
```